

ISTANBUL HEALTH AND TECHNOLOGY UNIVERSITY

Faculty of Engineering - Computer / Software Engineering Department

Group 24 Project Report

Design and Implementation of a Simple Two-Pass Compiler

System Programming Course - Spring 2025 / 2026

Group Members:

Haluk Alireisoğlu - 220609011

Sadık Ahmet Karabulut - 220609018

Repository: <https://github.com/S-Ahmet/TwoPassCompiler>

Demo Entry Point: `src/gui/Main.java`

1. System Design and Architecture

This project implements a simple two-pass compiler manually in Java. The compiler is designed for a small C-like teaching language and separates lexical analysis, parsing, semantic analysis, symbol table management, AST generation, and GUI presentation into clear modules.

Component	Responsibility
Lexer	Reads source code, skips whitespace/comments, creates tokens, detects lexical errors, and builds the symbol table.
Parser	Reads the token stream, validates grammar rules with recursive descent parsing, and builds an AST.
Semantic Analyzer	Checks declaration order, duplicate declarations, and expression/assignment type compatibility.
Symbol Table	Stores identifier name, type, scope, and memory location.
GUI	Allows users to write or load source code and displays tokens, symbol table, AST, errors, and line-token mapping.

2. Source Language Grammar (BNF)

<program>	::= <statement>*
<statement>	::= <declaration> <assignment> <if_statement> <while_statement> <print_statement>
<declaration>	::= <type> IDENTIFIER ";"
<type>	::= "int" "float"
<assignment>	::= IDENTIFIER "=" <expression> ";"
<if_statement>	::= "if" "(" <expression> ")" <block> ["else" <block>]
<while_statement>	::= "while" "(" <expression> ")" <block>
<print_statement>	::= "print" "(" <expression> ")" ";"
<block>	::= "{" <statement>* "}"
<expression>	::= <logical_or>
<logical_or>	::= <logical_and> (" " <logical_and>)*
<logical_and>	::= <equality> ("&&" <equality>)*
<equality>	::= <comparison> (("==" "!=") <comparison>)*
<comparison>	::= <term> (("<" ">" "<=" ">=") <term>)*
<term>	::= <factor> (("+" "-") <factor>)*
<factor>	::= <unary> (("*" "/") <unary>)*

```
<unary>          ::= ( "!" | "-" | "+" ) <unary> | <primary>
<primary>         ::= IDENTIFIER | INTEGER_LITERAL | FLOAT_LITERAL | STRING_LITERAL | "("
<expression>      ">"
```

3. Lexer Implementation (Pass 1)

The lexer scans the source code character by character and produces token objects containing type, value, and line number. It recognizes keywords, identifiers, integer and floating-point literals, string literals, operators, and delimiters.

- Whitespace is skipped.
- Single-line comments using `//` are skipped.
- Block comments using `/* ... */` are skipped.
- Malformed numbers, invalid characters, unclosed strings, and unclosed comments are reported as lexical errors with line numbers.
- The symbol table is built from valid variable declarations.

4. Parser and Semantic Analyzer (Pass 2)

The parser is a hand-written recursive descent parser. Expressions are parsed with precedence levels, so multiplication/division bind more tightly than addition/subtraction, and comparison/logical operators are handled at lower precedence levels.

The semantic analyzer processes tokens in source order and performs:

- Duplicate declaration detection.
- Use-before-declaration detection.
- Type inference for arithmetic, comparison, and logical expressions.
- Assignment compatibility checking.

The language allows assigning an integer expression to a float variable, but does not allow assigning a float, string, or boolean expression to an integer variable.

5. Symbol Table Structure

Field	Description
Name	Identifier name, such as <code>x</code> or <code>result</code> .
Type	Declared type: <code>int</code> or <code>float</code> .
Scope	Current implementation uses global scope.
Memory Location	Simulated memory address starting from 1000 and increasing by 4.

6. Error Handling Strategy

Error Type	Examples
Lexical	Invalid character, malformed number, unclosed string, unclosed block comment.
Syntax	Missing semicolon, missing parenthesis, unexpected token, malformed expression.
Semantic	Duplicate declaration, variable used before declaration, incompatible assignment type.

All error messages are displayed in the GUI Errors tab and include line numbers when available.

7. User Interface

The Java Swing GUI provides a source editor, a **LOAD FILE** button, and an **ANALYZE** button. The output is organized into five tabs:

- Tokens
- Symbol Table
- AST
- Errors
- Source Analysis

Screenshots can be added during final Blackboard upload if required by the instructor. The implementation already includes the GUI and all required output views.

8. Testing

The repository contains sample test files under the `samples` directory:

- `valid_program.txt`: valid source code with declarations, assignments, if/else, while, print, and comments.
- `lexical_error.txt`: malformed number and unclosed string.
- `syntax_error.txt`: malformed expression and missing parenthesis.
- `semantic_undeclared.txt`: assignment before declaration.
- `semantic_duplicate.txt`: duplicate variable declaration.
- `semantic_type_mismatch.txt`: assigning float expression to int variable.

9. Challenges and Solutions

Challenge	Solution
Correct operator precedence	Implemented recursive descent expression parsing with separate precedence functions.
Declaration-before-use checking	Semantic analyzer now scans tokens in source order.

Comment handling	Lexer skips both single-line and block comments before operator recognition.
Clear demo validation	Added sample source files for valid, lexical, syntax, and semantic cases.

10. Group Member Responsibilities

Student	Student Number	Responsibilities
Haluk Alireisoğlu	220609011	Testing, GUI validation, documentation, and final project delivery.
Sadık Ahmet Karabulut	220609018	Lexer, parser, semantic analyzer, symbol table, and repository implementation.